Relatório Final de Atividades

Extrator Toeplitz

Maurício Andrey

30 de outubro de 2026

Resumo

Análise a implementação serial de um extrator de entropia Toeplitz em FPGA, investigando seu funcionamento e desempenho por meio de simulações com Icarus Verilog. A partir da comparação com uma arquitetura de referência paralela, foram identificadas possíveis melhorias envolvendo a multiplicação por blocos, a geração de submatrizes e o uso de pipeline, visando reduzir a latência e melhorar o aproveitamento dos recursos de hardware.

Sumário

1 Contextualização e objetivo **3**

2 Análise da implementação **3**

3 Ambiente de simulação e adaptações **3**

4 Investigação de possíveis otimizações **3**

5 Possíveis avanços **4**

6 Como validar **4**

1 Contextualização e objetivo

O extrator de entropia Toeplitz realiza o condicionamento de uma sequência de bits, produzindo uma saída menor e próxima de uma distribuição uniforme, sob determinadas condições de entropia e independência da semente.

O objetivo é analisar o código atual do extrador e propor melhorias de desempenho em que as modificações não alterarem a função matemática do extrator, além de explorar o uso do paralelismo e de outros recursos da FPGA.

Foi investigado o código presente no repositório GitHub do link https://github.com/rokzitko/toeplitz, que foi clonado em outro repositório, com o link https://github.com/mauricioAndrey/toeplitz-verilog, para as análises e modificações.

2 Análise da implementação

Ferramentas Usadas : Icarus Verilog, GitHub.

Perguntas prévias

- Quantos ciclos são necessários para processar uma entrada?
  - Para uma quantidade N de bits de entrada tem K bits processados em paralelo por etapa, ele necessecita de 1 ciclo de clock para processar cada um dos K bits. Ou seja para uma entrada de N=256 bits e K=1 bit, ele precisa de 256 ciclos de clock. Sem considerar ciclos adicionais de inicialização, pipeline e finalização.
- Quais operações são realizadas em cada ciclo?
  - Os principais são as operações XOR e AND na multiplicação da matriz. Além da geração de coluna para fazer essa multiplicação.

3 Ambiente de simulação e adaptações

- Ambiente de simulação
  - Foi usado o Icarus Verilog (iverilog) para compilar os arquivos e gerar as saídas dos testbenches.
- Adaptações
  - Foram modificados os testbenches principais das implementações em série e em paralelo do toeplitz para poder analisar a quantidade de pulsos de clock.

4 Investigação de possíveis otimizações

Tendo como base o relatório técnico do ETSI desse ano.

Tabela 1: Comparação entre arquiteturas

Elemento	**Implementação serial**	**Arquitetura do artigo**
Entrada	1 bit por etapa	Vetor de entrada dividido em K bits
Geração da matriz	Uma coluna por etapa	Submatrizes correspondentes aos blocos de entrada
Multiplicação	Uma contribuição por bit de entrada	Multiplicação de cada submatriz pelo respectivo bloco
Acumulação	Atualização de 'y' a cada contribuição	XOR dos vetores intermediários
Organização	Processamento serial ao longo de N bits	Decomposição em blocos e processamento concorrente em pipeline

5 Possíveis avanços

- **Primeiro** - Usar a versão em paralelo que é parametrizada por K, com o K=1 (ou seja, sem paralelismo) para validar a equivalência funcional. Depois, aumentar progressivamente o K.
- **Segundo** - Implementar a geração de submatrizes: disponibilizar a submatriz correspondente às K colunas referentes ao bloco de K bits.
- **Terceiro** - Modificar a multiplicação e acumulação: recebe os vetores intermediários, gerados pela multiplicação da submatriz com o seu bloco de entrada correspondente, ao longo das etapas da geração das colunas.
- **Quarto** - Investigar a arquitetura pipeline: com a sequência

$$Matrix_building \rightarrow Submatrix_multiplication \rightarrow Vector_accumulation \quad (1)$$

  a ideia é permitir que a construção da próxima submatriz aconteça enquanto a multiplicação atual está sendo realizada, e que a acumulação aproveite os resultados anteriores.

  - O pipeline permite sobrepor o trabalho de diferentes estágios. Isso pode melhorar principalmente a taxa de processamento (throughput), enquanto a latência de uma operação individual pode continuar igual ou até aumentar.

6 Como validar

Critérios de avaliação e validação:

- **Equivalência funcional** - A saída da implementação modificada precisa ser idêntica à saída da implementação serial para os mesmos vetores.
- **Latência** - Verificar quantos ciclos decorrem entre o início do processamento e a disponibilização da saída.

- **Recursos e frequência** - Ver qual o impacto das modificações na quantidade de LUTs, flip-flops e frequência máxima.
